Spring MVC Security - Login Form Error Message - Overview and Coding.docx

In this lesson, we will focus on an essential aspect of user experience in web development—handling login error messages. You'll learn how to integrate this functionality into your custom login form using the Spring Security framework.

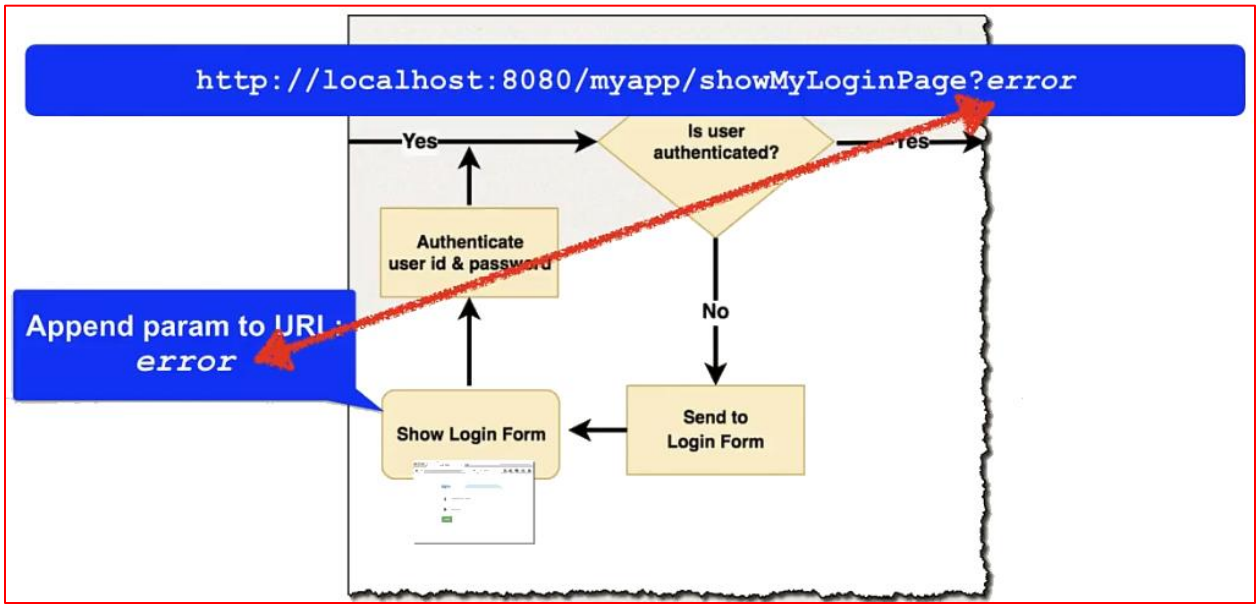
When using the default login form provided by Spring Security, error messages for failed login attempts are automatically displayed. But what if we want to create our own custom login form? You may have noticed that if a login attempt fails, the system simply redirects back to the login form without any error message. So how can we fix this?

Overview of Spring Security's Default Login Form

Spring Security's default login form comes with built-in support for displaying error messages. When you create a custom login form, you'll need to add additional code to handle failed logins. Our goal here is to display a message that informs the user, "*Hey, you entered an invalid username and password.*" We'll explore how to achieve that in this lecture.

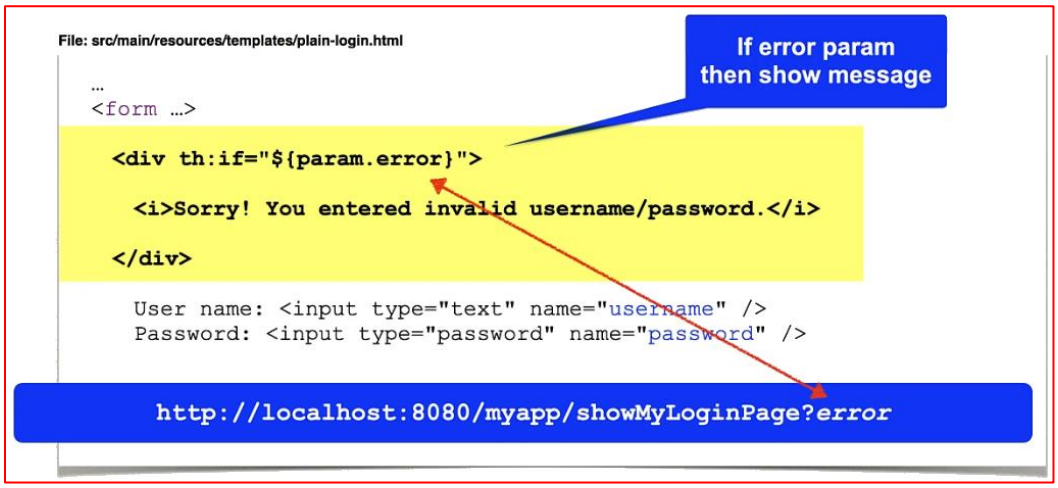
Workflow of a Failed Login

1. **Login Fails:** If authentication fails, Spring Security sends the user back to the login page and appends an error parameter, known as `?error`, to the URL.
2. **Check for Error Parameter:** We can write code to check for this parameter and respond accordingly.



Development Process

1. **Modify the Custom Login Form:** Add code to check for the error parameter.
2. **Show Error Message if Error Parameter Exists:** "Sorry, you entered an invalid username and password."



Step-by-Step Implementation

Step 1: Inside the block where you're handling the error parameter, add the following code:

```
"Sorry, you entered an invalid username and password."
```

This error message will be displayed if the error parameter is set, which Spring Security automatically does in the case of a failed login.

Step 2: Test the functionality by attempting to log in with incorrect credentials. Ensure that the quotes and curly braces are aligned correctly in your code.

Step 3: Implement a visual enhancement. Though the error message is functional, it's a bit plain. Let's apply some CSS to add color and style.

```
<!DOCTYPE html>
<html lang="en" xmlns:th="http://www.thymeleaf.org">
<head>
    <meta charset="UTF-8">
    <title>Custom Login Page</title>

    <style>
        .failed {
            color: red;
        }
    </style>
</head>
<body>
```

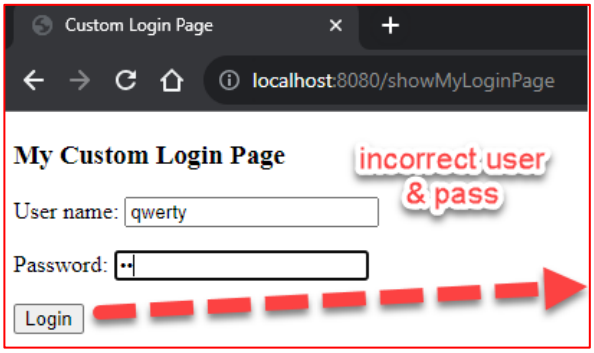
Then, in the HTML, apply the CSS style to the error message:

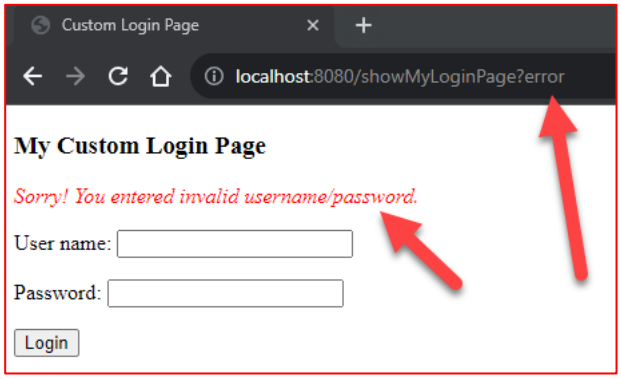
```
<span class="failed">Sorry, you entered the invalid username/password.</span>
```

```
<form action="#" th:action="@{/authenticateTheUser}" method="POST">

    <!-- Check for login error -->
    <div th:if="${param.error}">
        <i class="failed">Sorry! You entered invalid username/password.</i>
    </div>
```

Once you save these changes, test the application by opening a private window and attempting a login with incorrect credentials. The error message should now appear in red.





Summary

You've now added support for login error messages in your custom form. You've also had a chance to apply some basic CSS for visual enhancement.

Feel free to explore further and apply more advanced CSS styling or other frameworks. Remember, the key to effective web development lies not only in functionality but also in the design and user experience.